

Android Activity Lifecycle: Callbacks, Problems, and Fixes

Student Handout

Introduction

Every Android Activity passes through a series of callback methods as the user opens, leaves, and returns to it. Understanding these callbacks — and what breaks when they are ignored — is essential for writing apps that don't lose data, crash, leak memory, or drain the battery.

This handout has two parts:

1. **The six core lifecycle callbacks**, with code from a sample MainActivity.
2. **Six real problems** that occur when the lifecycle is ignored, each with the buggy code and the fix.

Part 1: The Six Lifecycle Callbacks

Sample MainActivity

```
package com.example.lifecycledemo;

import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "LifecycleDemo";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate() called");
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d(TAG, "onStart() called");
    }

    @Override
```

```

protected void onResume() {
    super.onResume();
    Log.d(TAG, "onResume() called");
}

@Override
protected void onPause() {
    super.onPause();
    Log.d(TAG, "onPause() called");
}

@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop() called");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d(TAG, "onDestroy() called");
}
}

```

Case 1: onCreate()

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Log.d(TAG, "onCreate() called");
}

```

Runs first, when the activity is created — either on first launch or after being previously destroyed (rotation, or being killed for memory). This is where you set the layout with `setContentView()`, initialize views, set up listeners, and restore saved state from `savedInstanceState`. It runs exactly once per activity instance and is followed immediately by `onStart()`.

Case 2: onStart()

```

@Override
protected void onStart() {
    super.onStart();
    Log.d(TAG, "onStart() called");
}

```

Called right after `onCreate()` (or after `onRestart()` when returning from a stopped state). The activity becomes visible here but is not yet interactive. Good place to start things that should run while visible, such as registering a `BroadcastReceiver`.

Case 3: onResume()

```
@Override
protected void onResume() {
    super.onResume();
    Log.d(TAG, "onResume() called");
}
```

Called after onStart(). The activity is now in the foreground and interactive. This is where you resume things paused earlier, like a camera preview or animations. The activity stays “resumed” until something interrupts it (a dialog, another activity, the home button).

Case 4: onPause()

```
@Override
protected void onPause() {
    super.onPause();
    Log.d(TAG, "onPause() called");
}
```

Called when the activity loses focus but is still partially visible (e.g. a dialog appears over it). Should run quickly — release resources like camera or sensors, save unsaved data. Avoid heavy work here since it blocks the next activity from resuming.

Case 5: onStop()

```
@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop() called");
}
```

Called when the activity is no longer visible at all. Release resources not needed while invisible (stop animations, unregister receivers registered in onStart()). From here the activity goes either to onRestart() → onStart() (user returns) or onDestroy() (system reclaims it).

Case 6: onDestroy()

```
@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d(TAG, "onDestroy() called");
}
```

The final callback before the activity is destroyed — from finish(), the user pressing Back, or the system reclaiming it. Clean up any remaining resources here (threads, listeners), since nothing runs after this for that instance.

Typical flow: onCreate → onStart → onResume (running) → onPause → onStop → onDestroy (or onRestart → onStart if the user returns before destruction).

Part 2: Six Problems When the Lifecycle Is Ignored

All six problems below share one root cause: the activity gets interrupted (a phone call, rotation, app switch, or low memory) and the code was written as if `onCreate()` runs exactly once, forever.

Problem 1: Data Lost on Rotation

Typed form input vanishes the moment the screen rotates.

Buggy code

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
}
```

`onCreate()` just inflates the layout every time, with no check for prior state. Rotation destroys this activity instance and builds a fresh one — the new `EditText` starts blank, so anything the user typed is gone.

Fix

```
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putString("draft_text", editText.getText().toString());
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        editText.setText(savedInstanceState.getString("draft_text"));
    }
}
```

Before destruction, Android calls `onSaveInstanceState()` automatically. `outState.putString(key, value)` writes the current text into the `Bundle`. On the new instance, that same `Bundle` comes back as `savedInstanceState`. The `!= null` check tells apart a fresh launch (`null`) from a recreation (`non-null`); if `non-null`, `getString("draft_text")` restores the text.

Problem 2: App Crashes

A late background callback updates a screen that no longer exists.

Buggy code

```
apiCall.enqueue(response -> {
    textView.setText(response.body());
});
```

`enqueue()` runs the network call on a background thread and invokes this lambda whenever the response arrives — possibly after the user has left and the activity was destroyed. Calling `setText()` on a `textView` from a dead activity throws a crash.

Fix

```
apiCall.enqueue(response -> {
    if (!isFinishing() && !isDestroyed()) {
        textView.setText(response.body());
    }
});
```

`isFinishing()` is true while the activity is closing (e.g. Back was pressed); `isDestroyed()` is true once it's fully gone. Checking both before touching the view means the update is skipped if the activity no longer exists, avoiding the crash.

Problem 3: Memory Leaks

The old Activity is held in memory and never freed.

Buggy code

```
public class MainActivity extends AppCompatActivity {
    static Context leakedContext;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        leakedContext = this;
    }
}
```

static fields belong to the class, not to any instance, and live as long as the app process runs — independent of any activity's lifecycle. `leakedContext = this` stores a reference to this Activity instance in that field. Even after `onDestroy()` runs, the static field still points to it, so garbage collection can never reclaim it. Every rotation leaks another full activity.

Fix

```
@Override
protected void onDestroy() {
    super.onDestroy();
    someListener.unregister();
}
```

`onDestroy()` is the last chance to clean up. Unregistering listeners (or nulling out static references) breaks the link keeping the old activity alive, letting the garbage collector free it.

Problem 4: Battery Drain

Camera, GPS, or sensors keep running unseen.

Buggy code

```
@Override
protected void onResume() {
    super.onResume();
    locationManager.requestLocationUpdates(...);
}
```

`requestLocationUpdates()` starts continuous GPS polling. It's started here with no matching call to stop it, so it keeps running even after the user leaves the screen — draining battery for updates nobody sees.

Fix

```
@Override
protected void onResume() {
    super.onResume();
    locationManager.requestLocationUpdates(...);
}

@Override
protected void onPause() {
    super.onPause();
    locationManager.removeUpdates(locationListener);
}
```

`onPause()` fires as soon as the activity stops being the foreground, interactive screen. `removeUpdates()` cancels the GPS subscription there, so tracking only runs while the screen is actually in front of the user — pairing every start with a stop.

Problem 5: Duplicate Actions

A form gets re-submitted, or data re-fetched, on every rotation.

Buggy code

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    fetchDataFromServer();
}
```

fetchDataFromServer() sits directly in onCreate(), which runs every time the activity is (re)created, including on rotation. A single rotation triggers a second network request or a duplicate submission.

Fix

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState == null) {
        fetchDataFromServer();
    }
}
```

savedInstanceState == null is only true on the very first creation. On any recreation after rotation it's non-null, so the fetch is skipped and doesn't repeat.

Problem 6: User Loses Progress

The app restarts from screen one after being killed in the background.

Buggy code

```
private int currentStep = 0;
```

This is a plain instance field, held only in RAM as part of the activity object. If Android kills the app process in the background to free memory, that object — and currentStep with it — is gone entirely. When the user returns, a new process starts and currentStep resets to its default, 0.

Fix

```
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("current_step", currentStep);
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        currentStep = savedInstanceState.getInt("current_step", 0);
    }
}
```

putInt() writes progress into the Bundle, which Android preserves separately from the process — it survives even a full process kill, not just rotation. getInt("current_step",

0) reads it back on recreation, with 0 as a fallback if the key is missing, restoring the user's place instead of starting over.

Summary Table

#	Problem	Cause	Fix
1	Data lost on rotation	No state saved before recreation	onSaveInstanceState() + restore in onCreate()
2	App crashes	Callback updates a dead view	Check isFinishing() / isDestroyed()
3	Memory leaks	Static field holds old Activity	Clear references in onDestroy()
4	Battery drain	Sensor/GPS started but never stopped	Pair onResume() start with onPause() stop
5	Duplicate actions	Network/fetch call placed in onCreate()	Guard with savedInstanceState == null
6	User loses progress	State kept only in a field, lost on process death	Persist via onSaveInstanceState()